

Complejidad Computacional: Espacio Polinomial (PSPACE)

Fundamentos Teóricos, Teoremas Clave y Resolución de la Práctica 5

Resumen Teórico-Práctico

Segundo Cuatrimestre

Índice

1 Fundamentos de Complejidad Espacial	1
1.1 Modelo de Cómputo y Medición del Espacio	1
1.2 Definición de PSPACE y NPSPACE	2
1.3 Cota de Configuraciones y Tiempo	3
2 Teoremas Estructurales de PSPACE	3
2.1 Teorema de Savitch	3
2.2 Clausura bajo Complemento	4
2.3 Teorema de la Jerarquía Espacial	4
3 Problemas Canónicos y PSPACE-Complejidad	5
3.1 Definición de PSPACE-Complejidad	5
3.2 Fórmulas Booleanas Cuantificadas y TQBF	5
3.3 Juegos de Dos Jugadores y Generalized Geography (GG)	6
4 Resolución Sistemática de la Práctica 5 (Parte PSPACE)	6
4.1 Ejercicio 1: Pertenencia a PSPACE	6
4.2 Ejercicio 2: Clausura por Complemento	9
4.3 Ejercicio 3: Demostración de $NP \subseteq PSPACE$	9
4.4 Ejercicio 4: Teorema de la Jerarquía Espacial	10
4.5 Ejercicio 5: Separación $P \neq SPACE[n]$ mediante Padding	11
4.6 Ejercicio 6: Generalized Geography (GG)	13
5 Cuadro Comparativo de Problemas	14
6 Resumen para el Examen: Puntos Críticos	14

1 Fundamentos de Complejidad Espacial

1.1 Modelo de Cómputo y Medición del Espacio

Para analizar el uso de memoria de manera rigurosa, se utiliza el modelo estándar de **Máquina de Turing (MT) con cintas de trabajo separadas**:

- **Cinta de entrada:** De solo lectura. La cabeza lectora puede moverse a izquierda y derecha pero no puede modificar las celdas.

- **Cintas de trabajo** ($k \geq 1$): De lectura y escritura.
- **Cinta de salida**: De solo escritura, donde la cabeza avanza unidireccionalmente.

Definición 1.1 (Espacio consumido por un cómputo). Sea M una máquina de Turing determinística o no determinística y sea $x \in \{0, 1\}^*$ una entrada. El **espacio utilizado por M con entrada x** es la cantidad máxima de celdas distintas visitadas en todas las cintas de trabajo a lo largo de la ejecución. La cinta de entrada de solo lectura y la cinta de salida no se contabilizan en el uso de espacio.

Definición 1.2 (Funciones construibles en espacio). Una función $S : \mathbb{N} \rightarrow \mathbb{N}$ se dice **construible en espacio** (space-constructible) si existe una máquina determinística que, recibiendo la cadena 1^n de longitud n , marca exactamente $S(n)$ celdas en su cinta de trabajo utilizando a lo sumo espacio $O(S(n))$. Todas las funciones usuales tales como $\log n$, n^c , 2^n son construibles en espacio.

Definición 1.3 (Clases SPACE y NSPACE). Sea $S : \mathbb{N} \rightarrow \mathbb{N}$. Definimos:

- $\text{SPACE}(S(n))$: Clase de los lenguajes decidibles por una MT determinística que para toda entrada de longitud n utiliza a lo sumo $O(S(n))$ celdas en sus cintas de trabajo.
- $\text{NSPACE}(S(n))$: Clase de los lenguajes decidibles por una MT no determinística donde toda rama de cómputo sobre entradas de longitud n utiliza a lo sumo $O(S(n))$ celdas en sus cintas de trabajo.

1.2 Definición de PSPACE y NPSPACE

Definición 1.4 (PSPACE y NPSPACE). Las clases de espacio polinomial determinístico y no determinístico se definen formalmente como:

$$\text{PSPACE} = \bigcup_{c>0} \text{SPACE}(n^c), \quad \text{NPSPACE} = \bigcup_{c>0} \text{NSPACE}(n^c)$$

Proposición 1.5 (Principio de Reutilización del Espacio). *A diferencia del tiempo de cómputo, que es acumulativo e irreversible, el espacio de memoria se puede borrar y reutilizar indefinidamente. Por lo tanto, un algoritmo puede ejecutar una cantidad exponencial de subrutinas sucesivas reutilizando siempre el mismo bloque polinomial de celdas.*

Proposición 1.6. $NP \subseteq \text{PSPACE}$.

Demostración. Sea $L \in NP$. Por definición, existe un verificador determinístico polinomial V y un polinomio $p(n)$ tal que para todo $x \in \{0, 1\}^*$:

$$x \in L \iff \exists u \in \{0, 1\}^{p(|x|)} \text{ tal que } V(x, u) = 1$$

Construimos una máquina determinística M' que decide L en espacio polinomial:

1. M' reserva un bloque de $p(|x|)$ celdas en una cinta de trabajo para almacenar el certificado candidato u .
2. Inicializa $u = 0^{p(|x|)}$.
3. En un bucle, simula la ejecución de $V(x, u)$.
4. Si $V(x, u)$ acepta, M' se detiene y acepta.
5. Si $V(x, u)$ rechaza, M' borra la memoria de trabajo utilizada durante la simulación de V , incrementa lexicográficamente la cadena u en el bloque reservado y vuelve al paso 3.

6. Si se agotan todos los $2^{p(|x|)}$ certificados sin éxito, M' rechaza.

Dado que V corre en tiempo polinomial $q(|x|)$, el espacio que consume en cada simulación está acotado por $q(|x|)$. Al limpiar y reutilizar el espacio entre iteraciones sucesivas, el espacio total en cualquier instante es:

$$\text{Espacio}(M') \leq p(|x|) + q(|x|) = O(|x|^c)$$

Por lo tanto, $L \in \text{PSPACE}$. □

1.3 Cota de Configuraciones y Tiempo

Definición 1.7 (Configuración de una Máquina de Turing). Una configuración de una MT M sobre entrada x en un instante dado queda unívocamente determinada por:

1. El estado interno actual $q \in Q$.
2. La posición del cabezal en la cinta de entrada ($0 \leq p_{\text{in}} \leq |x| + 1$).
3. El contenido de las $S(n)$ celdas utilizadas en las cintas de trabajo.
4. La posición de los cabezales en las cintas de trabajo ($0 \leq p_w \leq S(n)$).

Lema 1.8 (Cota de Configuraciones). Sea M una MT que utiliza a lo sumo $S(n)$ espacio de trabajo sobre entradas de tamaño n . Si el alfabeto de trabajo es Γ y $|Q|$ es la cantidad de estados, el número total de configuraciones globales distintas posibles está acotado por:

$$|C_M(x)| \leq |Q| \cdot (n+2) \cdot (S(n))^k \cdot |\Gamma|^{k \cdot S(n)} = 2^{O(S(n))} \quad (\text{para } S(n) \geq \log n)$$

Corolario 1.9 (Relación Espacio–Tiempo). Si una máquina determinística con espacio $S(n)$ no entra en un bucle infinito, su tiempo de ejecución está acotado por la cantidad total de configuraciones. Por ende:

$$\text{SPACE}(S(n)) \subseteq \text{DTIME}\left(2^{O(S(n))}\right) \implies \text{PSPACE} \subseteq \text{EXPTIME}$$

—

2 Teoremas Estructurales de PSPACE

2.1 Teorema de Savitch

El resultado más importante de la teoría de complejidad espacial establece que el no determinismo no añade un incremento super-polinomial cuando el recurso medido es el espacio.

Teorema 2.1 (Savitch, 1970). Para toda función construible en espacio $S(n) \geq \log n$:

$$\text{NSPACE}(S(n)) \subseteq \text{SPACE}\left(S(n)^2\right)$$

Esquema de Demostración. Sea $L \in \text{NSPACE}(S(n))$ decidido por una MT no determinística N . Para una entrada fija x de tamaño n , el grafo de configuraciones $G_{N,x}$ tiene a lo sumo $N_C = 2^{c \cdot S(n)}$ vértices. Preguntar si $x \in L$ equivale a decidir si existe un camino dirigido en $G_{N,x}$ desde la configuración inicial C_0 hasta alguna configuración de aceptación C_{acepta} de longitud máxima $M \leq 2^{c \cdot S(n)}$.

Se define el predicado recursivo por división y conquista $\text{Reach}(C_1, C_2, m)$, que devuelve verdadero si y solo si existe un camino de longitud a lo sumo 2^m desde C_1 hasta C_2 :

- **Caso base** ($m = 0$): $C_1 = C_2$ o $C_1 \vdash_N C_2$ en exactamente un paso válido de N .

- **Paso inductivo** ($m > 0$): Existe una configuración intermedia C_{medio} tal que:

$$\text{Reach}(C_1, C_{\text{medio}}, m-1) = \text{Verdadero} \quad \wedge \quad \text{Reach}(C_{\text{medio}}, C_2, m-1) = \text{Verdadero}$$

El algoritmo itera sobre cada configuración posible C_{medio} escribiéndola en memoria de trabajo. Para cada C_{medio} , evalúa primero $\text{Reach}(C_1, C_{\text{medio}}, m-1)$. Si esta llamada retorna 1, se descarta el espacio usado por esa subrutina y se evalúa $\text{Reach}(C_{\text{medio}}, C_2, m-1)$.

Análisis de Espacio:

- La profundidad máxima del árbol de recursión es $m = \lceil \log_2(2^{c \cdot S(n)}) \rceil = c \cdot S(n)$.
- En cada nivel del marco de activación de la pila solo se necesita guardar la configuración intermedia actual C_{medio} y los identificadores de C_1, C_2 , lo cual requiere $O(S(n))$ bits.
- El espacio se reutiliza entre llamadas hermanas de la recursión.

Por lo tanto:

$$\text{Espacio Total} = (\text{Profundidad}) \times (\text{Espacio por Nivel}) = O(S(n)) \cdot O(S(n)) = O(S(n)^2)$$

□

Corolario 2.2. $PSPACE = NPSPACE$.

Demostración. Tomando $S(n) = n^k$, por el Teorema de Savitch:

$$NPSPACE(n^k) \subseteq SPACE((n^k)^2) = SPACE(n^{2k}) \subseteq PSPACE$$

Tomando la unión sobre todo $k > 0$, obtenemos $NPSPACE \subseteq PSPACE$. Como la inclusión $PSPACE \subseteq NPSPACE$ es trivial por ser el determinismo un caso particular del no determinismo, se concluye la igualdad. □

2.2 Clausura bajo Complemento

Proposición 2.3. $PSPACE$ es cerrado bajo complemento, es decir:

$$PSPACE = coPSPACE$$

Demostración. Sea $L \in PSPACE$ decidido por una máquina determinística M en espacio $p(n)$ para algún polinomio p . Para evitar bucles infinitos en configuraciones repetidas, se dota a M de un contador binario en una cinta de trabajo separada que cuenta hasta $2^{c \cdot p(n)}$ pasos. Si el contador llega al límite sin alcanzar un estado de parada, la máquina sabe que ha entrado en un ciclo.

Para decidir $\bar{L}$, construimos una máquina M_{comp} que simula a M . Si M acepta, M_{comp} rechaza; si M rechaza (o excede la cota de pasos), M_{comp} acepta. El contador y la simulación usan espacio $O(p(n))$, que sigue siendo polinomial. □

2.3 Teorema de la Jerarquía Espacial

Teorema 2.4 (Jerarquía Espacial Determinística). Si $f, g : \mathbb{N} \rightarrow \mathbb{N}$ son funciones construibles en espacio que cumplen $f(n) = o(g(n))$ y $g(n) \geq \log n$, entonces:

$$SPACE(f(n)) \subsetneq SPACE(g(n))$$

Idea de Demostración por Diagonalización. Se define una máquina universal D que, sobre entrada $x = \langle M \rangle 10^*$, simula la ejecución de la máquina determinística M sobre la misma entrada x , utilizando un reloj espacial que marca exactamente $g(|x|)$ celdas en una cinta testigo.

1. D corre la simulación de $M(x)$ manteniéndose dentro de las $g(|x|)$ celdas acotadas.
2. Si $M(x)$ se detiene y acepta sin exceder el espacio, $D(x)$ rechaza.
3. Si $M(x)$ rechaza o intenta utilizar más de las $g(|x|)$ celdas asignadas (o entra en ciclo superando $2^{O(g(|x|))}$ pasos), $D(x)$ acepta.

Dado que simular una máquina de Turing de un alfabeto y número de cintas arbitrario sobre una máquina universal con cintas fijas se logra con un costo en espacio multiplicativo *constante* (a diferencia del factor logarítmico $\log T(n)$ que requiere la jerarquía temporal), la máquina D corre estrictamente en espacio $O(g(n))$, por lo que el lenguaje $L = L(D) \in \text{SPACE}(g(n))$.

Por el argumento de diagonalización usual, supongamos por el absurdo que $L \in \text{SPACE}(f(n))$. Entonces existe una máquina M_L que decide L en espacio a lo sumo $c \cdot f(n)$. Como $f(n) = o(g(n))$, existe un n_0 tal que para toda entrada x con $|x| \geq n_0$ se cumple $c \cdot f(|x|) < g(|x|)$. Al considerar una representación de M_L con suficiente padding tal que $|x| \geq n_0$, la máquina D tendrá espacio suficiente para simular completamente a $M_L(x)$ hasta el final. Entonces:

$$D(x) = 1 \iff M_L(x) = 0 \iff x \notin L$$

lo cual contradice que $L(D) = L$. En consecuencia, $\text{SPACE}(f(n)) \subsetneq \text{SPACE}(g(n))$. $\square$

Ejemplo 2.5. Como $n = o(n^2)$, tenemos que $\text{SPACE}(n) \subsetneq \text{SPACE}(n^2)$. En general, $\text{SPACE}(n^k) \subsetneq \text{SPACE}(n^{k+1})$ para todo $k \geq 1$.

—

3 Problemas Canónicos y PSPACE-Complejidad

3.1 Definición de PSPACE-Complejidad

Definición 3.1 (PSPACE-hard y PSPACE-completo). Un lenguaje $L \subseteq \{0, 1\}^*$ es:

- **PSPACE-hard:** Si para todo lenguaje $L' \in \text{PSPACE}$, se cumple que $L' \leq_p L$ (reducción de Karp en tiempo polinomial).
- **PSPACE-completo:** Si $L \in \text{PSPACE}$ y además L es PSPACE-hard.

3.2 Fórmulas Booleanas Cuantificadas y TQBF

Definición 3.2 (QBF - Quantified Boolean Formula). Una fórmula booleana cuantificada en forma prenexa tiene la estructura:

$$\Phi = Q_1 x_1 Q_2 x_2 \dots Q_m x_m \phi(x_1, \dots, x_m)$$

donde cada $Q_i \in \{\exists, \forall\}$ es un cuantificador y ϕ es una fórmula proposicional booleana sin cuantificadores sobre las variables $\{x_1, \dots, x_m\}$. Si la fórmula no posee variables libres, se denomina fórmula cerrada y tiene un valor de verdad definido: Verdadero (1) o Falso (0).

Problema 3.3 (TQBF). El lenguaje TQBF (True Quantified Boolean Formula) se define como:

$$\text{TQBF} = \{\langle \Phi \rangle : \Phi \text{ es una QBF cerrada y } \Phi \text{ es verdadera}\}$$

Teorema 3.4. TQBF es PSPACE-completo.

Esquema de Demostración. **1. Pertenencia a PSPACE:** Se define un algoritmo recursivo $\text{Eval}(\Phi)$:

- Si Φ no tiene cuantificadores, solo contiene constantes 0 y 1 conectadas por operadores booleanos. Se evalúa directamente en espacio y tiempo lineal respecto al tamaño de ϕ .

- Si $\Phi = \exists x \Psi(x)$, entonces $\text{Eval}(\Phi) = \text{Eval}(\Psi(0)) \vee \text{Eval}(\Psi(1))$.
- Si $\Phi = \forall x \Psi(x)$, entonces $\text{Eval}(\Phi) = \text{Eval}(\Psi(0)) \wedge \text{Eval}(\Psi(1))$.

El árbol de llamadas recursivas tiene profundidad igual a la cantidad de variables $m \leq |\Phi|$. En cada nivel solo se necesita almacenar el valor booleano del primer llamado mientras se evalúa el segundo. Al reutilizar el espacio de las ramas que ya terminaron, el espacio total consumido es $O(m \cdot |\Phi|) = O(|\Phi|^2)$, que es polinomial.

2. Dificultad (PSPACE-hard): Sea $L \in \text{PSPACE}$ decidido por una máquina determinística M en espacio $s(n) = n^c$. Para una entrada x , el cómputo de M tiene $2^{O(s(n))}$ configuraciones posibles de tamaño $O(s(n))$. Siguiendo la técnica del Teorema de Savitch, se construyen inductivamente fórmulas $\psi_i(C, C')$ que expresan que la configuración C' es alcanzable desde C en a lo sumo 2^i pasos. Para evitar que el tamaño de la fórmula crezca exponencialmente al desdoblar los dos subproblemas intermedios, se utiliza el cuantificador universal $\forall$:

$$\psi_i(C, C') \equiv \exists C_{\text{medio}} \forall (D_1, D_2) \in \{(C, C_{\text{medio}}), (C_{\text{medio}}, C')\} : \psi_{i-1}(D_1, D_2)$$

El tamaño de la fórmula resultante crece de forma estrictamente lineal respecto a la cantidad de pasos inductivos $O(s(n))$, produciendo una reducción computable en tiempo polinomial. $\square$

3.3 Juegos de Dos Jugadores y Generalized Geography (GG)

Una de las interpretaciones más naturales de PSPACE es la resolución de juegos de dos jugadores con información perfecta y turnos alternados:

- El cuantificador $\exists$ representa el turno del **Jugador 1**: “Existe una jugada ganadora”.
- El cuantificador $\forall$ representa el turno del **Jugador 2**: “Para toda respuesta del adversario, existe una contrarréplica...”.

Problema 3.5 (Generalized Geography (GG)). Dado un digrafo $G = (V, E)$ y un nodo inicial $v_0 \in V$:

- Inicialmente hay una ficha en v_0 .
- En cada turno, el jugador correspondiente debe mover la ficha a un vecino saliente no visitado previamente.
- El primer jugador que no tenga movimientos legales disponibles pierde.

$$\text{GG} = \{ \langle G, v_0 \rangle : \text{El Jugador 1 tiene una estrategia ganadora desde } v_0 \}$$

—

4 Resolución Sistemática de la Práctica 5 (Parte PSPACE)

A continuación se resuelven de forma exhaustiva todos los ejercicios de la guía de trabajos prácticos correspondientes a PSPACE.

4.1 Ejercicio 1: Pertenencia a PSPACE

Ejercicio 1 (1.a). Probar que $\text{CLIQUE} \in \text{PSPACE}$, donde:

$$\text{CLIQUE} = \{ \langle G, k \rangle : G \text{ contiene un subgrafo completo de } k \text{ vértices} \}$$

Solución 1.a. Presentamos dos formas equivalentes de demostrarlo:

Método 1 (Inclusión $NP \subseteq PSPACE$): Sabemos que CLIQUE $\in NP$, ya que dado un subconjunto $S \subseteq V$ de tamaño k como certificado, un verificador determinístico chequea en tiempo $O(k^2) \leq O(|V|^2)$ que para todo par de vértices $u, v \in S$ con $u \neq v$ exista la arista $(u, v) \in E$. Dado que $NP \subseteq PSPACE$, se concluye de forma inmediata que CLIQUE $\in PSPACE$.

Método 2 (Algoritmo Determinístico Directo en Espacio Polinomial): Diseñamos una máquina determinística M que sobre entrada $\langle G, k \rangle$, con $|V| = n$:

1. Reserva en una cinta de trabajo un vector binario de n bits para representar un subconjunto $S \subseteq V$.
2. Inicializa el vector en el primer subconjunto de tamaño k .
3. Para el subconjunto actual S , verifica si todos los pares de nodos en S son adyacentes en la matriz de adyacencia de G .
4. Si es una clique, la máquina acepta y termina.
5. Si no lo es, borra el resultado de la comprobación, avanza al siguiente subconjunto de tamaño k en orden lexicográfico y vuelve al paso 3.
6. Si se agotan los $\binom{n}{k} \leq 2^n$ subconjuntos posibles sin encontrar una clique, rechaza.

Análisis de Espacio: Guardar el subconjunto S requiere n bits. El chequeo de adyacencia requiere dos punteros de $\lceil \log_2 n \rceil$ bits para recorrer los vértices de S . Como el espacio se reutiliza entre combinaciones, el espacio total es $O(n)$, que es lineal y por lo tanto polinomial en el tamaño de la entrada. $\square$

Ejercicio 2 (1.b). Probar que FORMULA-MAS-CHICA $\in PSPACE$, donde:

$$\text{FORMULA-MAS-CHICA} = \{ \langle \phi, k \rangle : \phi \text{ es prop., y } \nexists \phi' \text{ tal que } \phi \equiv \phi' \text{ y } |\phi'| \leq k \}$$

Solución 1.b. Para probar que el lenguaje pertenece a PSPACE, alcanza con notar que $PSPACE = \text{coPSPACE}$. Por ende, basta con probar que el lenguaje complemento FORMULA-MAS-CHICA $\in PSPACE$.

Una entrada $\langle \phi, k \rangle$ pertenece al complemento si y solo si:

$$\exists \phi' \text{ tal que } |\phi'| \leq k \text{ y } \phi \equiv \phi'$$

Recordemos que dos fórmulas ϕ y ϕ' sobre un conjunto conjunto de variables $\{x_1, \dots, x_m\}$ son lógicamente equivalentes ($\phi \equiv \phi'$) si y solo si toman el mismo valor de verdad para las 2^m valuaciones booleanas posibles:

$$\phi \equiv \phi' \iff \forall v \in \{0, 1\}^m [\phi(v) = \phi'(v)]$$

Diseñamos un algoritmo determinístico en espacio polinomial para decidir este predicado:

1. La cantidad de fórmulas sintácticamente válidas ϕ' de longitud a lo sumo k sobre un alfabeto finito de símbolos lógicos y variables es finita (a lo sumo $|\Sigma|^k \leq 2^{O(k)}$).
2. Iteramos secuencialmente por cada fórmula candidata ϕ' de longitud $\leq k$, escribiéndola en una cinta de trabajo de tamaño $O(k)$.
3. Para cada ϕ' fija, verificamos si es equivalente a ϕ :
 - Inicializamos un contador $v \in \{0, 1\}^m$ en 0^m (donde $m \leq |\phi|$ es el número de variables).

- Evaluamos $\phi(v)$ y $\phi'(v)$. La evaluación de una fórmula booleana dada una valuación fija toma espacio proporcional a la profundidad del árbol sintáctico, que es $O(|\phi| + k)$.
 - Si $\phi(v) \neq \phi'(v)$, la fórmula ϕ' no es equivalente; descartamos ϕ' e incrementamos al siguiente candidato.
 - Si $\phi(v) = \phi'(v)$, incrementamos v . Si v recorre todas las 2^m valuaciones coincidiendo siempre con ϕ , hemos encontrado una fórmula equivalente de tamaño $\leq k$: el algoritmo **acepta**.
4. Si se prueban todas las fórmulas ϕ' de longitud $\leq k$ y ninguna es equivalente a ϕ , el algoritmo **rechaza**.

Análisis de Espacio:

- Almacenar la fórmula actual ϕ' requiere a lo sumo $k \leq |\langle \phi, k \rangle|$ celdas.
- El vector de valuación v ocupa $m \leq |\phi|$ bits.
- La evaluación de las fórmulas reutiliza un espacio de trabajo de tamaño $O(|\phi| + k)$.

Dado que $k \leq n$, el espacio total requerido en cualquier instante de la ejecución es $O(n)$, donde $n = |\langle \phi, k \rangle|$. Por lo tanto, el problema está en PSPACE. $\square$

Ejercicio 3 (1.c). Probar que TA-TE-TI_k $\in$ PSPACE, donde el tablero es una matriz T de $n \times n$ con entradas $\{0, 1, 2\}$ y se desea decidir si el jugador 1 tiene una estrategia ganadora para conectar k en línea.

Solución 1.c. El problema modela un juego de suma cero entre dos jugadores con información perfecta sobre un tablero finito de tamaño $n \times n$.

1. **Cota de turnos:** Dado que el tablero tiene n^2 casilleros y en cada turno una celda vacía (valor 0) pasa a ser ocupada por una ficha de valor 1 o 2 sin moverse ni removerse jamás, cualquier partida termina en a lo sumo n^2 jugadas.
2. **Algoritmo Minimax / DFS:** Diseñamos una función recursiva determinística GanaJugador1(T , turno)
 - Si el tablero T ya contiene una configuración ganadora para el Jugador 1, retorna **Verdadero**.
 - Si el tablero T tiene una configuración ganadora para el Jugador 2 o no quedan casilleros libres, retorna **Falso**.
 - Si es el turno del **Jugador 1**:

$$\text{GanaJugador1}(T, 1) = \bigvee_{\text{casillero libre } c} \text{GanaJugador1}(T \cup \{c \mapsto 1\}, 2)$$

- Si es el turno del **Jugador 2**:

$$\text{GanaJugador1}(T, 2) = \bigwedge_{\text{casillero libre } c} \text{GanaJugador1}(T \cup \{c \mapsto 2\}, 1)$$

Análisis de Espacio:

- La profundidad máxima del árbol de recursión está estrictamente acotada por el número total de casilleros, es decir, n^2 .
- En cada nivel del árbol de recursión, solo necesitamos almacenar el estado del tablero actual modificado o las coordenadas del casillero explorado, lo que toma $O(n^2)$ bits.

- Al recorrer el árbol en profundidad (DFS), el espacio de una rama se libera y reutiliza para explorar la siguiente jugada hermana.

Por lo tanto:

$$\text{Espacio total} = (\text{Profundidad}) \times (\text{Espacio por marco}) = n^2 \times O(n^2) = O(n^4)$$

Como $O(n^4)$ es polinomial respecto al tamaño de la entrada, TA-TE-TI_k ∈ PSPACE. □

4.2 Ejercicio 2: Clausura por Complemento

Ejercicio 4. Probar que PSPACE está cerrado por complemento.

Demostración Formal. Sea $L \in \text{PSPACE}$. Por definición, existe una máquina de Turing determinística M y una constante $c > 0$ tal que M decide L en espacio acotado por $S(n) = n^c$.

Para garantizar la parada absoluta frente a cualquier entrada, consideramos la cota de configuraciones: existe una constante $d > 0$ tal que la cantidad de configuraciones posibles de M sobre una entrada de tamaño n es a lo sumo $2^{d \cdot n^c}$.

Construimos una máquina de Turing determinística M' para el lenguaje complemento $\bar{L} = \{0, 1\}^* \setminus L$:

1. M' reserva dos cintas de trabajo:
 - La cinta 1 para simular paso a paso las cintas de trabajo de M .
 - La cinta 2 para implementar un contador binario de longitud $d \cdot n^c + 1$ bits, inicializado en 0.
2. En cada paso de la simulación de M , incrementa el contador de la cinta 2 en una unidad.
3. Si M alcanza el estado de aceptación q_{acepta} , M' se detiene y entra a su estado de rechazo q'_{rechaza} .
4. Si M alcanza el estado de rechazo q_{rechaza} , M' se detiene y entra a su estado de aceptación q'_{acepta} .
5. Si el contador de pasos alcanza el valor $2^{d \cdot n^c}$ sin que M se haya detenido, por el principio del palomar M repitió al menos una configuración y entró en un bucle infinito de rechazo. En ese caso, M' interrumpe la simulación, se detiene y entra a su estado de aceptación q'_{acepta} .

Consumo de Espacio:

- La simulación en la cinta 1 consume $S(n) = n^c$ celdas.
- El contador de la cinta 2 utiliza exactamente $O(n^c)$ celdas.

El espacio total utilizado por M' es $O(n^c)$, lo cual es polinomial. En consecuencia, $\bar{L} \in \text{PSPACE}$, demostrando que $\text{PSPACE} = \text{coPSPACE}$. □

4.3 Ejercicio 3: Demostración de $\text{NP} \subseteq \text{PSPACE}$

Ejercicio 5. Probar formalmente que $\text{NP} \subseteq \text{PSPACE}$.

Demostración Formal. Sea $L \in \text{NP}$. Por caracterización por verificador polinomial, existen una máquina de Turing determinística V y polinomios p, q tales que para todo $x \in \{0, 1\}^*$:

$$x \in L \iff \exists u \in \{0, 1\}^{p(|x|)} \text{ tal que } V(x, u) \text{ acepta en a lo sumo } q(|x|) \text{ pasos}$$

Construimos una máquina determinística M que decide L :

1. Dada una entrada x con $n = |x|$, M calcula $p(n)$ y reserva en su cinta de trabajo 1 un bloque de $p(n)$ celdas.
2. Escribe en dicho bloque la cadena $u = 0^{p(n)}$.
3. Simula la ejecución del verificador $V(x, u)$ utilizando una cinta de trabajo 2.
4. Si $V(x, u)$ acepta, entonces M acepta la entrada x y se detiene inmediatamente.
5. Si $V(x, u)$ rechaza:
 - M borra completamente el contenido de la cinta 2 (reutilización de espacio).
 - Si $u = 1^{p(n)}$ (se probaron todas las combinaciones), M rechaza y se detiene.
 - En caso contrario, reemplaza el contenido de la cinta 1 con el sucesor lexicográfico de u y vuelve al paso 3.

Consumo de Espacio:

- La cinta 1 almacena el certificado u , ocupando exactamente $p(n)$ celdas.
- La cinta 2 simula el verificador V . Como V se detiene en a lo sumo $q(n)$ pasos, el cabezal de V no puede visitar más de $q(n)$ celdas en toda la ejecución.

Por ende, el espacio total utilizado por M en todo instante es:

$$S_M(n) \leq p(n) + q(n)$$

Como la suma de polinomios es un polinomio, $S_M(n) = O(n^k)$ para alguna constante k . Por lo tanto, M decide L en espacio polinomial determinístico, concluyendo que $L \in \text{PSPACE}$. $\square$

4.4 Ejercicio 4: Teorema de la Jerarquía Espacial

Ejercicio 6. Probar que si $f, g : \mathbb{N} \rightarrow \mathbb{N}$ son funciones construibles en espacio que satisfacen $f(n) = o(g(n))$, entonces:

$$\text{SPACE}(f(n)) \subsetneq \text{SPACE}(g(n))$$

Demostración Formal. La inclusión $\text{SPACE}(f(n)) \subseteq \text{SPACE}(g(n))$ es inmediata del hecho de que $f(n) \leq g(n)$ para todo n suficientemente grande. Para probar que la inclusión es estricta, construiremos un lenguaje $L_{\text{diag}} \in \text{SPACE}(g(n))$ tal que $L_{\text{diag}} \notin \text{SPACE}(f(n))$ mediante el método de diagonalización.

Fijemos una enumeración efectiva de todas las máquinas de Turing determinísticas $M_1, M_2, \dots$ donde cada máquina tiene una codificación binaria $\langle M_i \rangle$.

Definimos una máquina de Turing determinística D que opera de la siguiente manera ante una entrada $w \in \{0, 1\}^*$ con $n = |w|$:

1. Utilizando la hipótesis de que g es construible en espacio, D computa la cota $g(n)$ y marca exactamente $g(n)$ celdas en una cinta de control de espacio. Si en algún momento subsiguiente el cómputo intenta sobrepasar esta marca, el proceso se aborta inmediatamente.
2. D parsea la entrada w . Si w no tiene la forma $\langle M \rangle 10^*$, D rechaza. Si tiene la forma indicada, identifica a la máquina M .
3. D simula la ejecución de M sobre la entrada w . Dado que la simulación universal de una máquina de Turing sobre otra con número fijo de cintas solo requiere un factor constante de memoria extra c_M , D mantiene la simulación dentro del límite de $g(n)$ celdas disponibles.

4. Además, para evitar bucles infinitos, D mantiene un contador en una cinta auxiliar que cuenta hasta $2^{g(n)}$ pasos.
5. Si la simulación de $M(w)$ termina dentro de las $g(n)$ celdas antes de que expire el contador:
 - Si $M(w)$ acepta, entonces $D(w)$ **rechaza**.
 - Si $M(w)$ rechaza, entonces $D(w)$ **acepta**.
6. Si $M(w)$ intenta usar más de $g(n)$ celdas o el contador de pasos expira, $D(w)$ se detiene y **rechaza**.

Pertenencia a $\text{SPACE}(g(n))$: Por construcción, la máquina D jamás utiliza más de $O(g(n))$ celdas en sus cintas de trabajo, de modo que $L(D) \in \text{SPACE}(g(n))$.

No pertenencia a $\text{SPACE}(f(n))$: Supongamos por contradicción que $L(D) \in \text{SPACE}(f(n))$. Entonces existe alguna máquina determinística M_k que decide $L(D)$ utilizando a lo sumo $c \cdot f(n)$ celdas para alguna constante $c > 0$.

Como $f(n) = o(g(n))$, se cumple por definición de límite que:

$$\lim_{n \rightarrow \infty} \frac{c \cdot f(n)}{g(n)} = 0$$

Existe entonces un umbral $n_0 \in \mathbb{N}$ tal que para todo $n \geq n_0$:

$$c \cdot f(n) < g(n)$$

Elijamos una cadena de entrada $w = \langle M_k \rangle 10^m$ eligiendo la cantidad de ceros m suficientemente grande para que $|w| \geq n_0$.

Al ejecutar $D(w)$, la máquina D simula a $M_k(w)$. Dado que $|w| \geq n_0$, el espacio que M_k necesita para decidir w es a lo sumo $c \cdot f(|w|) < g(|w|)$. Por ende, la simulación de $M_k(w)$ cabe holgadamente dentro del espacio delimitado $g(|w|)$ y no se aborta por espacio ni entra en ciclo. Por la regla de diagonalización de D :

$$D(w) \text{ acepta} \iff M_k(w) \text{ rechaza}$$

Sin embargo, por hipótesis M_k decide el lenguaje de D , lo que impone:

$$D(w) \text{ acepta} \iff M_k(w) \text{ acepta}$$

Llegamos a una contradicción flagrante ($1 \iff 0$). La suposición de que $L(D) \in \text{SPACE}(f(n))$ es falsa. Consecuentemente:

$$\text{SPACE}(f(n)) \subsetneq \text{SPACE}(g(n))$$

□

4.5 Ejercicio 5: Separación $P \neq \text{SPACE}[n]$ mediante Padding

Ejercicio 7. Probar que $P \neq \text{SPACE}[n]$.

Demostración Formal. Demostraremos este resultado por reducción al absurdo utilizando la técnica de **padding (relleno)**.

Paso 1: Suposición de la igualdad y propiedad de clausura de P . Recordemos que la clase P está cerrada bajo polinomios: si un lenguaje A se puede decidir en tiempo polinomial respecto a $|x|^2$, entonces $A \in P$, porque un polinomio evaluado en un polinomio sigue siendo un polinomio:

$$(n^2)^k = n^{2k} \implies \bigcup_{k>0} \text{DTIME}((n^2)^k) = P$$

Paso 2: Construcción por Padding. Sea L un lenguaje arbitrario en $\text{SPACE}[n^2]$. Definimos el lenguaje con padding cuadrático L_{pad} como:

$$L_{\text{pad}} = \{x01^{|x|^2-|x|-1} : x \in L\}$$

Para cualquier palabra $w \in L_{\text{pad}}$, su longitud es $N = |w| = |x|^2$.

Paso 3: Pertenencia de L_{pad} a $\text{SPACE}[N]$. Diseñamos una máquina M_{pad} para decidir si $w \in L_{\text{pad}}$:

1. Recibe w de longitud N . Verifica si tiene el formato $x01^*$ con $N = |x|^2$. Esto requiere contar longitudes, lo cual toma espacio logarítmico $O(\log N)$.
2. Simula la máquina M_L que decide L sobre la subcadena x .

Como $L \in \text{SPACE}[n^2]$, la simulación de $M_L(x)$ consume a lo sumo $c \cdot |x|^2$ celdas de espacio. Pero como la entrada tiene longitud $N = |x|^2$, el espacio utilizado es:

$$\text{Espacio} \leq c \cdot |x|^2 = c \cdot N = O(N)$$

Por lo tanto:

$$L_{\text{pad}} \in \text{SPACE}[N] = \text{SPACE}[n]$$

Paso 4: Derivación del Absurdo. Supongamos ahora por contradicción que $P = \text{SPACE}[n]$.

1. Como $L_{\text{pad}} \in \text{SPACE}[n]$, por la hipótesis del absurdo se deduce que $L_{\text{pad}} \in P$.
2. Por ende, existe un algoritmo determinístico que decide si $w \in L_{\text{pad}}$ en tiempo $O(N^k)$ para alguna constante k .
3. Construimos un algoritmo para decidir la palabra original $x \in L$:
 - Recibe x con $n = |x|$.
 - Construye la cadena $w = x01^{n^2-n-1}$ de longitud $N = n^2$. Escribir esta cadena toma tiempo $O(n^2)$.
 - Ejecuta el algoritmo polinomial de L_{pad} sobre w .

El tiempo total de este procedimiento sobre x es:

$$\text{Tiempo} = O(n^2) + O(N^k) = O(n^2) + O((n^2)^k) = O(n^{2k})$$

Como n^{2k} es polinomial en n , concluimos que $L \in P$.

Como L era un lenguaje **arbitrario** de $\text{SPACE}[n^2]$, esto implicaría que:

$$\text{SPACE}[n^2] \subseteq P$$

Sin embargo, por nuestra hipótesis del absurdo teníamos $P = \text{SPACE}[n]$. Reemplazando obtenemos:

$$\text{SPACE}[n^2] \subseteq \text{SPACE}[n]$$

lo cual contradice directamente el **Teorema de la Jerarquía Espacial** (Teorema 2.3), ya que $n = o(n^2)$ y por lo tanto $\text{SPACE}[n] \subsetneq \text{SPACE}[n^2]$.

La contradicción provino de suponer que $P = \text{SPACE}[n]$. Concluimos indefectiblemente que:

$$P \neq \text{SPACE}[n]$$

□

4.6 Ejercicio 6: Generalized Geography (GG)

Ejercicio 8. Probar que el juego Generalized Geography (GG) pertenece a PSPACE.

Demostración Formal. Recordemos la definición del problema: dado un grafo dirigido $G = (V, E)$ y un nodo distinguido $v \in V$, dos jugadores mueven una ficha alternadamente a lo largo de las aristas dirigidas sin poder repetir vértices ya visitados. El juego termina cuando el jugador de turno no tiene movimientos válidos hacia nodos no visitados, perdiendo la partida.

Sea $n = |V|$ la cantidad de vértices del grafo.

1. **Cota estricta de turnos:** Como ningún vértice puede ser visitado más de una vez, una partida puede tener a lo sumo n turnos. El árbol de juego tiene una profundidad máxima de n .
2. **Algoritmo Recursivo:** Diseñamos una función $\text{EstrategiaGanadora}(u, S)$, donde $u \in V$ es la posición actual de la ficha y $S \subseteq V$ es el conjunto de vértices ya visitados hasta el momento:
 - Sea $N_{\text{libres}}(u) = \{w \in V : (u, w) \in E \wedge w \notin S\}$ el conjunto de vecinos legales no visitados.
 - Si $N_{\text{libres}}(u) = \emptyset$, el jugador actual no puede mover y pierde; la función retorna **Falso**.
 - En caso contrario, el jugador actual tiene al menos un movimiento. Gana si y solo si *existe* algún movimiento a un vértice $w \in N_{\text{libres}}(u)$ tal que el oponente *no* tenga estrategia ganadora desde w :

$$\text{EstrategiaGanadora}(u, S) = \bigvee_{w \in N_{\text{libres}}(u)} \neg \text{EstrategiaGanadora}(w, S \cup \{w\})$$

Para decidir si $\langle G, v_0 \rangle \in \text{GG}$, la máquina determinística invoca:

$$\text{EstrategiaGanadora}(v_0, \{v_0\})$$

Análisis del Espacio de Memoria:

- La recursión se resuelve por búsqueda primero en profundidad (DFS).
- La profundidad de la pila de llamadas es a lo sumo n .
- En cada nivel i de la pila de llamadas ($1 \leq i \leq n$), solo se necesita guardar:
 - El nodo actual u ($\lceil \log_2 n \rceil$ bits).
 - El conjunto de visitados S , que puede representarse como una máscara de n bits.
 - El índice del vecino w que se está explorando actualmente ($\lceil \log_2 n \rceil$ bits).
- Al retornar de una llamada recursiva, el espacio utilizado para explorar el subárbol se descarta y reutiliza para el siguiente vecino w' .

Por lo tanto, el consumo total de espacio en cualquier instante de la ejecución es:

$$\text{Espacio Total} = (\text{Profundidad Máxima}) \times (\text{Espacio por Nivel}) = n \times O(n) = O(n^2)$$

Como $O(n^2)$ es polinomial en el tamaño de la entrada, se concluye que:

$$\text{GG} \in \text{PSPACE}$$

□

5 Cuadro Comparativo de Problemas

La siguiente tabla resume la clasificación precisa de los problemas analizados en este bloque de estudio:

Problema	Clase de Complejidad	Técnica / Justificación
CLIQUE	NP-completo ($\in$ PSPACE)	Verificador poli. / Recorrido lexicográfico
3-SAT	NP-completo ($\in$ PSPACE)	Verificación de asignaciones / Iteración $\in$ SPACE(n)
FORMULA-MAS-CHICA	PSPACE ($\in$ coPSPACE)	Fuerza bruta evaluando equivalencia en espacio po
TA-TE-TI _k ($n \times n$)	PSPACE	Minimax / DFS acotado en profundidad n^2
TQBF	PSPACE-completo	Inducción con $\forall$ y $\exists$ (Savitch en fórmulas)
Generalized Geography	PSPACE-completo	Partidas sin ciclos acotadas por $ V $ turnos

Cuadro 1: Resumen de problemas y su pertenencia en la jerarquía espacial.

6 Resumen para el Examen: Puntos Críticos

1. ¿Cómo justificar que un problema está en PSPACE?

- Si está en P o NP, decir: "Está en NP, y como $NP \subseteq PSPACE$, entonces pertenece a PSPACE".
- Si es un juego o problema de optimización combinatoria: diseñar un algoritmo recursivo (DFS / backtracking) y probar que:

$$\text{Espacio} = (\text{Profundidad del árbol}) \times (\text{Espacio por nivel}) \leq \text{Polinomial}$$

- Si es más fácil pensar un algoritmo no determinístico, invocar el Teorema de Savitch: "Existe una MTND que lo decide en espacio poli, por ende está en NPSPACE = PSPACE".

- Cierre por complemento:** $PSPACE = coPSPACE$. Invertir respuestas de aceptación y rechazo jamás altera el espacio utilizado.
- Teorema de la Jerarquía Espacial:** Si $f(n) = o(g(n))$, entonces $SPACE(f(n)) \subsetneq SPACE(g(n))$. No hay pérdidas logarítmicas de simulación.
- Padding:** Sirve para separar clases con distintas propiedades de clausura polinomial (por ejemplo, P es cerrada bajo polinomios mientras que $SPACE[n]$ no lo es, lo cual fuerza $P \neq SPACE[n]$).